CYBER SEC — CAU EDR OPERATIONS GUIDE

Download, Deploy & Kernel Feature Flags

An in-depth, start-to-finish reference for the CAU EDR engine: obtaining the software, building and installing it, deploying the on-host agent, and configuring every kernel feature flag in the cau-kernel enforcement backends (BPF-LSM and kernel module). Includes command explanations, error meanings, and fixes for common problems.

Product	CAU EDR — engine + agent (MIT, free, unmetered)
Kernel sensor	cau-kernel — MIT userspace / GPL-2.0 BPF program
Theme	Sentinel Blue — Cyber Sec site design system
Fonts	Chakra Petch · IBM Plex Sans · JetBrains Mono (self-hosted WOFF2)
Page size	A4 · full-bleed dark · running header/footer
Source	/home/kali/cau · /home/kali/Projects/site

GUIDE CONTENTS

- Download & build
- Install & deploy
- Package managers
- Verify installation
- Resource discovery
- cau-kernel kernel flags
- BPF-LSM & kmod backends
- Posture, baseline & drift
- Attestation (TPM / IMA / kdump)
- CLI reference & flags
- Detection content & formats
- Er… troub…

Contents

1. Overview

2. Download

3. Build from source

4. Install & deploy

5. Package managers

6. Verify installation

7. Resource discovery

8. cau-kernel — kernel feature flags

8.1 Overview & backends

8.2 Kernel requirements & flags

8.3 BPF-LSM backend

8.4 Kernel-module backend

8.5 Enforce command flags

8.6 Network-shell guard

8.7 Path-aware policy

8.8 Kill / ptrace guard

8.9 Posture sysctls

8.10 Baseline & drift

8.11 Attestation

9. CAU engine CLI reference

9.1 Commands

9.2 Common flags

9.3 Exit codes

10. Detection content & output formats

11. Errors & troubleshooting

12. Honest limits & disclaimer

What CAU is

CAU EDR ("Certs Are Useless") is a transparent open-source malware detection engine in Go. Fifteen detectors read every file once; every finding is explained with a labeled class and an ATT&CK technique; every report is machine-readable with CI exit codes. Three runtime planes (process, network, persistence) are joined by cau defend into a single incident surface. The kernel-level anti-tamper guards live in the sibling cau-kernel project.

Two artifacts

cau (this repository, MIT) — the engine and agent: all detection content and scanning, the MCP server, the evaluation harness, and all prevention and response on the host (quarantine, restore, block, kill, canaries). Detection and prevention are free and unmetered, forever.

cau-kernel (MIT userspace / GPL-2.0 BPF program) — kernel-integrity sensor: read-only detection of kernel drift, boot attestation, and opt-in execution-freeze enforcement.

How this guide is organized

Follow the sections in order: download the software, build it, install it, verify it works, then configure the kernel feature flags if you need enforcement. Each command block shows what the command does, what its flags mean, what can go wrong, and how to fix it.

Download

Obtain the CAU engine and the cau-kernel sensor from source, release tarballs, or distro packages.

01 Clone the repositories

```
$ git clone https://github.com/s-b-repo/cau.git cau
$ cd cau
$ git clone https://github.com/s-b-repo/cau-kernel.git cau-kernel
```

Cloning downloads the full source history. You need the cau repository (engine + agent) and cau-kernel (kernel-integrity sensor). Clone cau-kernel inside or alongside cau; they are independent Go modules.

Error: repository not found / authentication required
Meaning: You mistyped the URL, the repo is private, or you lack network access to GitHub.
Fix: Verify the URL is `https://github.com/s-b-repo/cau.git`. If behind a proxy, set `git config --global http.proxy`. If the repo requires auth, generate a GitHub personal access token and use `https://<token>@github.com/s-b-repo/cau.git`.

02 Or download a release tarball

Every v tag triggers CI, which builds cau-tag-linux-amd64.tar.gz with the rules and signature databases, plus SHA256SUMS. Download from https://github.com/s-b-repo/cau/releases/tag/v0.1.0 (first tag v0.1.0, cut 2026-11-02) or build from source.

Error: 404 or file not found on release page
Meaning: The tag does not exist yet, or the CI release job has not finished uploading artifacts.
Fix: Check https://github.com/s-b-repo/cau/releases for the latest tag. Wait for the CI release job to complete (green checkmark), then download the tarball and its SHA256SUMS file.

Signed releases

Release artifacts carry an Ed25519-signed update bundle (hashes.txt + manifest.json + manifest.json.sig) and a signed SHA256SUMS. Verify before installing: cau verify-self --from dir checks the binary against the signed manifest. Never run an unsigned bundle.

SOURCE	COMMAND / LOCATION	NOTES
Source (Git)	git clone https://github.com/s-b-repo/cau.git cau	Engine + agent, MIT
Kernel sensor (Git)	git clone https://github.com/s-b-repo/cau-kernel.git cau-kernel	Kernel guards, MIT userspace
Release tarball	cau-tag-linux-amd64.tar.gz	Rules + signatures attached to the GitHub release
Debian / Kali .deb	cau_version-1_amd64.deb	From GitHub release or signed apt repo
Arch PKGBUILD	packaging/arch or yay -S cau	Builds from vendored module tree
macOS	brew install go yara pkg-config	Then build from source

Build from source

Requirements, toolchain, and the build commands.

DEPENDENCY	VERSION	WHY
Go	1.24 or newer	The engine is a single Go module.
libyara (with headers)	4.3 or newer	YARA runs in-process via go-yara; the yara binary is not needed.
pkg-config	any	go-yara locates libyara through it.
C compiler	any	cgo, for the libyara binding.

01 Install dependencies

```
# Debian / Ubuntu / Kali
sudo apt install golang-go libyara-dev pkg-config build-essential
# Fedora
sudo dnf install golang yara-devel pkgconf-pkg-config gcc
# Arch Linux
sudo pacman -S go yara pkgconf
# macOS
brew install go yara pkg-config
```

These packages provide the Go compiler, the libyara development headers (required because YARA runs in-process through go-yara), pkg-config so Go can find libyara, and a C compiler for cgo.

Error: package yara was not found in the pkg-config search path
Meaning: pkg-config cannot locate libyara, usually because libyara-dev is not installed or is in a non-standard prefix.
Fix: Install libyara-dev (or yara-devel on Fedora). If installed under ~/~/.local, source `scripts/dev-env.sh` or export `PKG_CONFIG_PATH=$HOME/.local/lib/pkgconfig:$PKG_CONFIG_PATH`.

Error: go command not found
Meaning: Go is not installed or not on PATH.
Fix: Install Go (golang-go on Debian/Kali, golang on Fedora, go on Arch, brew install go on macOS). Verify with `go version`.

Error: cc1: error: cannot find -lyara / undefined reference to yr_*
Meaning: The C compiler cannot link libyara; headers may be present but the shared library is missing or not on the linker path.
Fix: Ensure libyara-dev is installed (provides both headers and the shared library). If in a custom prefix, export `LD_LIBRARY_PATH` and `CGO_LDFLAGS=-L/your/lib/path`.

02 Build the engine

```
git clone https://github.com/s-b-repo/cau.git cau
cd cau
make build          # → ./cau  (version stamped from git describe)
./cau version
```

`make build` runs `go build` and stamps the binary version from `git describe` (the latest tag). Plain `go build .` also works but reports `0.1.0-dev` because no tag was found.

Error: make: * No rule to make target 'build'. Stop.**
Meaning: You are not in the cau repository root, or the Makefile is missing.
Fix: Run `cd cau` first, then confirm `ls Makefile` exists. If you cloned into a subdirectory, adjust the path.

Error: module cau not found / go: cannot find module
Meaning: Go cannot locate the module, usually because you are outside the module root or vendor/ is missing.
Fix: Run from the repository root where go.mod lives. If vendor/ is expected but missing, run `go mod vendor` (requires network) or ensure the vendored tree is committed.

03 User-local toolchain (no root)

If Go or libyara live under your home directory, scripts/dev-env.sh exports PATH, PKG_CONFIG_PATH, LD_LIBRARY_PATH and CGO_ENABLED=1 for ~/.local/go and ~/.local/yara:

```
source scripts/dev-env.sh
make build
```

This lets you build without sudo when tools are installed to ~/.local rather than system paths.

Error: scripts/dev-env.sh: Permission denied
Meaning: The script is not executable.
Fix: Run `chmod +x scripts/dev-env.sh` then source it again.

04 Build cau-kernel

```
cd cau-kernel
go build -o cau-kernel .        # pure Go, standard library only
```

cau-kernel is pure Go with no cgo, so it needs no libyara or C compiler. The default binary has no eBPF dependency and reports enforcement unavailable.

Error: build constrained / build tags not satisfied

Meaning: You need the ebpf build tag for enforcement (make enforce) but built without it.

Fix: For the BPF-LSM enforcer, run `make enforce` which compiles the BPF object and builds with `-tags ebpf` (needs clang). The default `go build -o cau-kernel .` intentionally omits ebpf.

Install & deploy

Install the binary, rules, signatures, man pages, completions, and optional systemd units.

01 Install system-wide

```
sudo make install                  # PREFIX=/usr/local by default
```

Copies the cau binary, cau-mcp server, YARA rules, signature databases, man pages, license, docs, and shell completions to /usr/local by default. Requires root because it writes to system directories.

Error: install: cannot create directory '/usr/local/...': Permission denied
Meaning: You ran make install without sudo, or PREFIX points to a directory you cannot write.
Fix: Prefix with sudo, or use a user-local prefix: `make install PREFIX=$HOME/.local`.

02 Install user-local (no root)

```
make install PREFIX=$HOME/.local
```

Installs everything under your home directory so no root is needed. The binary goes to $HOME/.local/bin — make sure that directory is on your PATH.

Error: cau: command not found after user-local install
Meaning: $HOME/.local/bin is not on your PATH.
Fix: Add `export PATH=$HOME/.local/bin:$PATH` to your shell rc file (~/.bashrc, ~/.zshrc), then run `source ~/.bashrc` or log out and back in.

03 Optional systemd units

```
sudo make install-systemd PREFIX=/usr
sudo make install-config           # /etc/cau/cau.conf
sudo make install-logrotate        # /etc/logrotate.d/cau
```

Installs the systemd service/timer units, the agent configuration file at /etc/cau/cau.conf, and the logrotate policy for cau-defend's log. These are optional; cau works without them.

04 Shell completion

```
cau completion bash  > /etc/bash_completion.d/cau
cau completion zsh   > /usr/share/zsh/site-functions/_cau
cau completion fish  > /usr/share/fish/vendor_completions.d/cau.fish
```

Prints shell completion scripts. Redirect to the appropriate completions directory for your shell so tab-completion works for cau subcommands and flags.

Error: cau completion: command not found
Meaning: The cau binary is not installed or not on PATH.
Fix: Complete the install step first, or run `./cau completion bash` from the build directory.

PATH	CONTENT
$PREFIX/bin/cau	The engine binary
$PREFIX/bin/cau-mcp	The MCP server
$PREFIX/share/cau/rules/*.yar	Bundled YARA rules
$PREFIX/share/cau/signatures/hashes.txt	Hash signature database
$PREFIX/share/cau/signatures/fuzzy.txt	Fuzzy-hash database (empty by design)
$PREFIX/share/man/man1/cau*.1	Man pages
$PREFIX/share/licenses/cau/LICENSE	MIT license
$PREFIX/share/doc/cau/	Documentation, README, changelog
bash / zsh / fish completions	Shell completion scripts

cau-defend service

cau defend runs as cau-defend.service and reads /etc/cau/cau.conf. A command-line flag wins over the file. It defends by default. Set `report-only = true` to watch a host first. A host that must never act should put `--report-only` on the unit's ExecStart line, because the command line wins over the config file.

Package managers

The fastest supported install path. Every package is built, installed and smoke-tested by CI on Debian, Kali Linux and Arch Linux.

Debian and Kali Linux

```
# GitHub release asset
sudo apt install ./cau_version-1_amd64.deb

# Or add the signed repository
curl -fsSL https://host/pubkey.asc | sudo tee /usr/share/keyrings/cau.asc
echo 'deb [signed-by=/usr/share/keyrings/cau.asc] https://host stable main' | sudo tee /etc/apt/
sources.list.d/cau.list
sudo apt update && sudo apt install cau
```

Install a downloaded .deb directly with apt, or add the signed apt repository to receive updates through apt. Replace `<host>` with your repository host.

Error: unable to locate package cau / E: Repository not signed
Meaning: The repository key is not trusted, or apt's package list does not include cau.

Fix: Import the pubkey with the curl command above, run `sudo apt update` to refresh package lists, then install. For a direct .deb, ensure you are installing the correct architecture (amd64) and run `sudo apt install ./cau_*.deb` from the download directory.

Error: dpkg: error processing cau (--install): dependency problems
Meaning: The .deb depends on libyara or golang packages not yet installed.

Fix: Run `sudo apt install -f` to fix broken dependencies, then retry `sudo apt install ./cau_*.deb`.

Build the .deb from source

```
sudo apt install build-essential debhelper golang-go libyara-dev pkg-config
dpkg-buildpackage -us -uc          # needs a checkout with vendor/ committed
sudo apt install ../cau_*_amd64.deb
```

Build a Debian package from a source checkout. The vendored module tree must be committed so the build never touches the Go module proxy.

Error: dpkg-buildpackage: cannot find go module / vendor missing
Meaning: The checkout does not have vendor/ committed.

Fix: Run `go mod vendor` and commit vendor/ before building the package, or ensure you are building from a tagged release that includes it.

Arch Linux

```
cd packaging/arch && makepkg -si        # from a checkout
yay -S cau                              # from the AUR
```

Build from the packaging/arch directory with makepkg, or install from the AUR with yay. The PKGBUILD builds from the committed vendored module tree and stamps the version from the git tag.

Error: makepkg: cannot find go / missing dependencies
Meaning: go or base-devel is not installed.

Fix: Install `go` and `base-devel` (or the equivalent build tools), then retry makepkg -si.

What the package does not do for you

cau selftest refuses to trust content it cannot verify and exits 3 until an integrity manifest has been recorded with cau selftest --record. That is documented behaviour, not a broken install: the manifest that ships in a release is the one cau selftest verifies against.

06 · VERIFY

Verify the installation

Confirm the binary runs and detects a known sample.

```
cau scan --file corpus/smoke/malicious/mimikatz_strings.bin --db corpus/smoke/hashes.txt
echo "exit $?"          # 1: a YARA rule matched
make eval               # validate the smoke corpus, run it, gate against the baseline
```

What this does

Scans a known-malicious sample (mimikatz strings binary) against the smoke hash database. A YARA rule should match, returning exit code 1 (findings). `make eval` runs the full smoke corpus evaluation harness and gates results against the baseline.

Error: cau: command not found
Meaning: cau is not installed or not on PATH.

Fix: Complete the install step, or run the binary directly from the build directory: `./cau scan ...`.

Error: could not open file corpus/smoke/...: no such file or directory
Meaning: You are not in the cau repository root, or the smoke corpus is missing.

Fix: Run `cd cau` first. Confirm `ls corpus/smoke/malicious/mimikatz_strings.bin` exists.

Expected output

```
[!] DETECTED (YARA) — corpus/smoke/malicious/mimikatz_strings.bin
    Verdict   : MALICIOUS
    [YARA] default::mimikatz_strings
           author=CAU threat_level=5 in_the_wild=true desc="…"
           $m1 @ 0x48 : "sekurlsa::logonpasswords"
[+] Fusion — corpus/smoke/malicious/mimikatz_strings.bin
    Verdict   : MALICIOUS (fused 1.14 / 1.00, high confidence)
    Agreed    : yara, heuristic
    Techniques: T1003
```

Exit codes

0 clean · 1 findings at the --fail-on level · 2 unscannable files · 3 the run failed · 4 usage.

Resource discovery

The binary looks for its signature database, rules directory and fuzzy database in this order and uses the first that exists:

1. The flag (--db, --rules, --fuzzy-db)
2. The environment (CAU_SIGNATURES, CAU_RULES, CAU_FUZZY)
3. Next to the binary (dir of cau/signatures/hashes.txt, dir of cau/rules)
4. Share directory relative to the binary (dir of cau/../share/cau/...)
5. /usr/local/share/cau and /usr/share/cau
6. The current working directory

Missing resources

When nothing is found, the error lists every path that was tried. A missing signature database is fatal; missing rules or a missing fuzzy database turn that detector off with a message on stderr.

Error: signature database not found (fatal)

Meaning: cau could not locate hashes.txt in any of the six discovery locations.

Fix: Pass `--db /path/to/hashes.txt` explicitly, set `CAU_SIGNATURES=/path/to/hashes.txt` in the environment, or install cau so the database lands in /usr/share/cau/signatures/hashes.txt.

Error: rules not found / YARA disabled

Meaning: No YARA rule directory was found, so the YARA detector is turned off.

Fix: Pass `--rules /path/to/rules` or set `CAU_RULES`. This is non-fatal — cau continues with other detectors.

cau-kernel — kernel feature flags

The kernel-integrity sensor for CAU. Two halves: read-only detection (userspace Go) and opt-in enforcement (BPF-LSM guard or kernel module).

Detection (default binary — userspace, read-only)

Pure Go, standard library only, no cgo, no kernel component. Reads /proc and /sys under a configurable root, summarises kernel integrity-relevant state, and diffs against a signed baseline. Cannot change anything. Deploy widely, run from a timer.

- **Modules** — /proc/modules vs /sys/module cross-view (hidden-module reconciliation), refcounts, dependents.
- **Taint** — /proc/sys/kernel/tainted, decoded (E unsigned module, O out-of-tree, F force-loaded, K live-patched, D OOPS/BUG).
- **eBPF surface** — bpffs mounts, pinned maps/programs, bpf_jit_harden, unprivileged_bpf_disabled sysctls.
- **Kernel memory** — /dev/mem, /dev/kmem, /dev/port, /proc/kcore, lockdown.
- **Posture** — hardening sysctls, lockdown mode, active LSM list (/sys/kernel/security/lsm), IMA presence.

Enforcement (opt-in, privileged)

Two interchangeable backends implement the same primitive — an execution freeze: while armed, every new execve is denied. Both are fail-open: if the resident enforcer dies, the attachment drops and execution resumes, so a bug can never permanently brick the host.

	BPF-LSM (bpf/, default)	Kernel module (kmod/)
Mechanism	lsm/bprm_check_security eBPF program	kretprobe on security_bprm_check
Power	Verifier-bounded; LSM hooks only	Arbitrary in-kernel C; any kprobe
Safety	Cannot crash the kernel; auto-detaches	Can panic; taints the kernel
Build	-tags ebpf (cilium/ebpf), needs clang	Out-of-tree module, DKMS, kernel headers
Default?	Yes	Only where BPF-LSM cannot serve

Kernel requirements & feature flags

What the running kernel must provide, and the sysctl knobs cau-kernel reads and reports on.

BPF-LSM requirements

- Kernel with CONFIG_BPF_LSM
- bpf present in /sys/kernel/security/lsm
- Root, or CAP_BPF + CAP_MAC_ADMIN
- clang (to compile the BPF object)

Error: enforcement unavailable

Meaning: The default cau-kernel binary was built without -tags ebpf, or the kernel lacks BPF-LSM support.

Fix: Rebuild with `make enforce` (needs clang). Verify the kernel has `CONFIG_BPF_LSM` and that `bpf` appears in `cat /sys/kernel/security/lsm`. If not, use the kernel-module backend instead.

Error: bpf not in /sys/kernel/security/lsm

Meaning: The kernel does not have BPF-LSM registered as an LSM.

Fix: You need a kernel built with CONFIG_BPF_LSM and LSM=...,bpf,... in the kernel boot parameters or config. Otherwise use the kmod backend.

Posture sysctls (cau-kernel posture)

Each knob is marked ok or weak with the reason it matters. State, not scolding — the operator owns the policy call.

SYSCTL	HARDENED	WHY
kernel.kptr_restrict	≥ 1	Kernel pointer exposure via /proc
kernel.dmesg_restrict	1	Unprivileged kernel-log access
kernel.kexec_load_disabled	1	Loading a new kernel over the running one
kernel.modules_disabled	1	Further module loading (one-way latch)
kernel.unprivileged_bpf_disabled	≥ 1	Unprivileged eBPF attack surface
net.core.bpf_jit_harden	≥ 1	BPF JIT-spray hardening
kernel.yama.ptrace_scope	≥ 1	Cross-process ptrace (credential theft)
kernel.perf_event_paranoid	≥ 2	perf_event_open exposure

Taint flags (decoded from /proc/sys/kernel/tainted)

FLAG	MEANING	SECURITY-RELEVANT?
E	Unsigned module loaded	Yes
O	Out-of-tree module loaded	Yes
F	Force-loaded module	Yes
R	Force-unloaded module	Yes
K	Live-patched kernel	Yes
D	Kernel OOPS / BUG	Yes
W	Warning (taint warning)	No
X	External (out-of-tree) module	No
t	Unprivileged BPF	No
n	Retpoline	No
u	Upper 32-bit / upper 64-bit	No

BPF-LSM backend (default)

Built with the ebpf tag (make enforce). Attaches to lsm/bprm_check_security and pins the control map to /sys/fs/bpf/cau-kernel.

01 Build the enforcer

```
cd cau-kernel
make enforce              # compile the BPF object + build with -tags ebpf
```

Compiles the BPF object (bpf/lsm.bpf.o or similar) with clang and builds the cau-kernel binary with the -tags ebpf build tag so it can load the eBPF program.

Error: clang: command not found
Meaning: clang is not installed, required to compile BPF objects.
Fix: Install clang (sudo apt install clang on Debian/Kali).

Error: BPF verifier rejected program / invalid argument
Meaning: The eBPF program failed kernel verifier checks (too complex, unsafe access).
Fix: Use a newer kernel with a more permissive verifier, or simplify the BPF program. This is rare with the shipped guard.

02 Run the resident guard

```
sudo ./cau-kernel enforce run      # attach the guard, state = allow
```

Attaches the BPF-LSM program to lsm/bprm_check_security and pins the control map to bpffs. The guard is resident and starts in allow state (executions permitted). Requires root or CAP_BPF + CAP_MAC_ADMIN.

Error: operation not permitted / permission denied
Meaning: You lack CAP_BPF + CAP_MAC_ADMIN or are not root.
Fix: Run with sudo, or grant the capabilities: `sudo setcap cap_bpf,cap_mac_admin+ep ./cau-kernel`.

Error: no such file or directory /sys/fs/bpf/cau-kernel
Meaning: bpffs is not mounted at /sys/fs/bpf.
Fix: Mount bpffs: `sudo mount -t bpf bpf /sys/fs/bpf`.

03 Freeze / release execution

```
sudo ./cau-kernel enforce on       # freeze: deny every new execve
sudo ./cau-kernel enforce off      # release
./cau-kernel enforce status        # report current state
```

`on` flips the pinned control map to deny all new execve (execution freeze). `off` releases it. `status` reads the pinned map and reports allow or deny.

Error: control map not found / map not pinned
Meaning: enforce run has not been executed, or the resident guard died and detached.
Fix: Start the guard first with `sudo ./cau-kernel enforce run`, then use on/off/status.

> **Fail-open, by design**
> run holds the LSM attachment and pins the control map to bpffs; on/off/status flip or read that pinned map. If run dies, the program detaches and execution is no longer frozen — a crash can never brick the host. The attachment lives only as long as run.

Kernel-module backend

More capable than eBPF (arbitrary in-kernel C, no verifier, hooks eBPF cannot reach) but heavier and less safe: it can panic and taints the kernel. Use only where BPF-LSM cannot serve.

01 Install kernel headers

```
sudo apt-get install -y linux-headers-$(uname -r)
```

Installs the kernel headers matching your running kernel, required to build an out-of-tree module.

Error: package linux-headers-$(uname -r) not found
Meaning: Headers for your exact kernel version are unavailable in the repos.
Fix: Update and reboot into a kernel for which headers exist, or install the generic headers package (linux-headers-generic).

02 Build the module

```
make -C kmod                        # build cau_guard.ko
```

Builds cau_guard.ko from the kmod directory using the kernel build system.

Error: make: * /lib/modules/$(uname -r)/build: No such file or directory**
Meaning: Kernel headers are not installed or the build symlink is broken.
Fix: Install linux-headers-$(uname -r) and ensure /lib/modules/$(uname -r)/build points to the kernel source.

03 Load and freeze

```
sudo insmod kmod/cau_guard.ko
echo 1 | sudo tee /sys/module/cau_guard/parameters/enforce   # freeze
```

Loads the module into the kernel, then sets the enforce parameter to 1, which freezes all new execve via a kretprobe on security_bprm_check returning -EPERM.

Error: insmod: ERROR: could not insert module cau_guard: Required key not available
Meaning: Secure Boot is enabled and the module is not signed by an enrolled key.
Fix: Sign the module with an enrolled MOK key (see kmod/README.md for DKMS packaging and Secure Boot signing), or disable Secure Boot in firmware settings (not recommended for production).

Error: insmod: ERROR: could not insert module cau_guard: Invalid argument
Meaning: The module is tainted or incompatible with the running kernel.
Fix: Ensure you built against the exact running kernel headers. The module taints the kernel (expect an O/E taint flag).

04 Release and unload

```
echo 0 | sudo tee /sys/module/cau_guard/parameters/enforce   # release
sudo rmmod cau_guard
```

Sets enforce to 0 (release the freeze), then removes the module from the kernel.

Error: rmmod: ERROR: Module cau_guard is in use
Meaning: Something still references the module.
Fix: Ensure enforce is set to 0 first, then retry rmmod. If a process holds it, identify and stop that process.

Mechanism

Kretprobes security_bprm_check and overrides the return to -EPERM while enforce is set. DKMS packaging and Secure Boot signing are in kmod/README.md. Under Secure Boot the module must be signed by an enrolled MOK key or the kernel refuses it.

Enforce command reference

All cau-kernel enforce subcommands.

COMMAND	EFFECT
enforce run [--core obj]	Resident: attach the guard (state = allow). With --core, load a CO-RE object for path-aware policy.
enforce on	Freeze: deny every new execve.
enforce off	Release the freeze.
enforce status	Report current state (reads the pinned control map).
enforce block path	Deny opening that file (keyed on device + inode).
enforce unblock path	Remove the block.
enforce blocked	List blocked files.
enforce killguard --obj obj	Resident: attach kill + ptrace guard hooks.
enforce protect --pid pid	Defend a process from kill/ptrace.
enforce unprotect --pid pid	Remove protection.
enforce protected	List protected PIDs.
enforce netshell [--mode audit|deny] [--any-program audit|deny] [--memfd audit|deny] [--log file]	Network-shell guard (see next section).

Network-shell guard flags

lsm_netshell.bpf.c fires on every execve (lsm/bprm_check_security) before the new image runs. A program whose stdin and stdout are both an established AF_INET/AF_INET6 socket is a reverse or bind shell.

01 Build and run in audit mode

```
make enforce
sudo ./cau-kernel enforce netshell                # audit everything, print JSON events
```

Starts the network-shell guard in audit mode: it logs every program on network stdio without blocking anything. Use this first to see what would be caught.

02 Enable deny mode

```
sudo ./cau-kernel enforce netshell --mode deny      # refuse interpreters on network stdio
sudo ./cau-kernel enforce netshell --mode deny --log /run/cau-kernel.jsonl
```

`--mode deny` refuses interpreters (shells, Python, Perl, Ruby, PHP, Lua, Node, awk) whose stdin and stdout are both network sockets. `--log` writes JSON events for later correlation.

Start in audit

`deny` refuses a legitimate inetd-style service as readily as a reverse shell, and `--memfd deny` breaks runc, which re-executes through a memfd. Blind spots: a shell on a pty, a pipe relay (mkfifo | sh | nc), and a shell started first and connected afterwards by an in-process implant. cau ps catches the relay from userspace.

03 Correlate with live processes

```
cau ps --kernel-events /run/cau-kernel.jsonl        # correlate with live processes
```

Joins the guard's JSON events with live process data from cau ps so you can see which process triggered which network-shell event.

FLAG	DEFAULT	APPLIES TO
--mode	audit	Interpreters on network stdio (shells, Python, Perl, Ruby, PHP, Lua, Node, awk)
--any-program	audit	Every other program on network stdio (catches cp /bin/bash /tmp/x)
--memfd	audit	Execution from a memfd (fileless path, nothing on disk records)

Error: --memfd deny breaks runc

Meaning: runc re-executes through a memfd, so denying memfd execution breaks container runtimes.

Fix: Keep `--memfd audit` unless you are certain no container runtime runs on this host.

Path-aware (per-inode) blocklist

Load the CO-RE object (make -C bpf lsm_fileopen.bpf.o, no libbpf-dev needed) to add per-file denial on top of the global freeze. Files are keyed on device and inode.

01 Run with path-aware core object

```
sudo ./cau-kernel enforce run --core bpf/lsm_fileopen.bpf.o   # resident, path-aware
```

Loads the CO-RE lsm/file_open object alongside the freeze guard, enabling per-file denial keyed on device and inode.

Error: CO-RE object not found / failed to load
Meaning: bpf/lsm_fileopen.bpf.o was not built, or the kernel lacks CO-RE support (BTF).
Fix: Run `make -C bpf lsm_fileopen.bpf.o` to build it. Ensure the kernel has BTF (CONFIG_DEBUG_INFO_BTF) for CO-RE to work.

02 Block / unblock a file

```
sudo ./cau-kernel enforce block   /path/to/implant           # deny opening that file
sudo ./cau-kernel enforce unblock /path/to/implant
```

Resolves a path to its device+inode and flips the cau_blocked_inodes pinned map. block denies opening that specific file; unblock restores access.

Error: path not found / cannot resolve inode
Meaning: The file does not exist or is on a filesystem where inode resolution fails.
Fix: Use an absolute path to an existing file. block refuses system paths and interpreters (e.g. /bin/bash) to prevent bricking the system.

03 List blocked files

```
./cau-kernel enforce blocked                                  # list blocked files
```

Lists all files currently in the blocked-inodes map.

How it works

run --core loads lsm/file_open and pins both the freeze flag and the cau_blocked_inodes map. block/unblock resolve a path to its inode and flip that pinned map. cau ps --respond stop --block-exe drives this for a confirmed implant and refuses to block a system path or an interpreter.

Kill / ptrace guard (self-protection)

Protects the sensor itself — stops malware from killing or hooking the EDR (ATT&CK T1562.001, Impair Defenses). Hooks lsm/task_kill (deny every signal, including SIGKILL/SIGSTOP) and lsm/ptrace_access_check (deny debugger/injector attach).

01 Build the killguard object

```
make -C bpf core                                          # builds lsm_killguard.bpf.o
```

Builds the CO-RE BPF object containing both lsm/task_kill and lsm/ptrace_access_check hooks.

02 Attach both hooks

```
sudo ./cau-kernel enforce killguard --obj bpf/lsm_killguard.bpf.o  # resident: attach both hooks
```

Attaches the killguard program as a resident process; it pins the cau_protected_pids map.

03 Protect the sensor

```
sudo ./cau-kernel enforce protect   --pid "$(pgrep -x cau)"        # defend the sensor's PID
```

Adds cau's PID to the protected-pids map, so signals and ptrace attaches to it are denied.

04 Remove protection / list protected

```
sudo ./cau-kernel enforce unprotect --pid 1234
./cau-kernel enforce protected                                     # list protected PIDs
```

Remove a PID from protection, or list all currently protected PIDs.

Honest limits

- **Fail-open.** Kill the resident killguard (it is not itself protected) and all protection lifts. protect refuses PID 1 and its own PID so the guard can never be made un-teardownable.
- **User-space signals only.** Some kernel-internal paths (e.g. the OOM killer) do not pass through security_task_kill.
- **Real kernel only.** Needs CONFIG_BPF_LSM and bpf in /sys/kernel/security/lsm, as root. CI only proves it compiles.

cau-kernel posture

Just the hardening posture, each knob marked ok or weak. Run read-only, any time.

```
cau-kernel inventory      # full snapshot of kernel state the sensor watches
cau-kernel posture        # just the hardening posture
```

What posture reports

kptr_restrict, dmesg_restrict, kexec_load_disabled, modules_disabled, unprivileged_bpf_disabled, bpf_jit_harden, yama.ptrace_scope, perf_event_paranoid, lockdown mode, the active LSM list, and whether IMA is present. Some nodes (IMA, kprobes) need root; unreadable nodes become warnings, never silent gaps.

Error: permission denied reading /sys/kernel/security/ima/

Meaning: IMA measurement log requires root; running as unprivileged user degrades to a warning.

Fix: Run with sudo for full IMA visibility, or accept the warning — posture still reports what it can read.

Baseline & drift detection

Record a signed baseline from a known-good boot, then detect drift on a schedule.

01 Record a signed baseline

```
cau-kernel baseline --out /var/lib/cau-kernel/baseline.json --sign-key priv.hex
```

Creates a signed snapshot of current kernel state. The --sign-key is an Ed25519 private key (hex). Store the baseline on a known-good host.

02 Detect drift

```
cau-kernel drift --baseline /var/lib/cau-kernel/baseline.json --pubkey pub.hex
```

Compares current kernel state against the signed baseline. Exits non-zero on security-relevant changes: new module, new taint flag, new kernel-memory channel, new pinned eBPF object, hardening regression (ok → weak), or hidden-module suspect.

drift flags

- --baseline file — signed baseline to compare against
- --pubkey file — Ed25519 public key for fail-closed verification
- --all — show info-level (benign) changes too
- --format text\|json — output format
- --root path — run against a fixture tree (default /)

The baseline is Ed25519-signed; without --pubkey a tampered baseline is still detected as drift, and with --pubkey a tampered baseline fails closed.

Error: baseline signature does not verify
Meaning: The baseline file was modified after signing, or the wrong public key was supplied.

Fix: Regenerate the baseline with the correct --sign-key, and verify with the matching --pubkey. Do not edit baseline.json by hand.

Error: drift exit code non-zero (alert)
Meaning: A security-relevant change was detected since baseline.

Fix: Review the drift output. If the change is expected (legitimate update), record a new baseline. If unexpected, treat as a potential compromise: contain the host, capture evidence, and consider cau-kernel enforce on to freeze execution.

Boot attestation (TPM / IMA / kdump)

Records and verifies boot-anchored measurement state — things a running kernel cannot rewrite retroactively — against a signed baseline.

01 Show current state

```
cau-kernel attest show                                  # print current state
```

Prints current TPM PCR values, IMA measurement count/violations, and kdump readiness.

02 Record a signed baseline

```
cau-kernel attest baseline --out a.json --sign-key priv.hex
```

Signs the current boot measurements. Record this on a trusted boot, ideally right after provisioning.

03 Verify against baseline

```
cau-kernel attest verify   --baseline a.json --pubkey pub.hex
```

Compares current PCRs, IMA state, and kdump readiness against the signed baseline. A changed PCR means the boot chain differs (different kernel, initrd, bootloader config, or Secure Boot state).

What attest reads

- **TPM PCRs** (/sys/class/tpm/tpm0/pcr-sha256/*) — measured-boot registers. World-readable via sysfs.
- **IMA** (/sys/kernel/security/ima/) — runtime measurement count, appraisal violation count, SHA-256 digest of the measurement log.
- **kdump readiness** (/sys/kernel/kexec_crash_loaded, kexec_crash_size) — whether a crash kernel is reserved and loaded.

Error: TPM not found / no pcr-sha256 directory
Meaning: The host has no TPM, or TPM sysfs is not available.

Fix: Attestation reports PCR absence as a warning; attest still works using IMA and kdump where available.

Error: attest verify exit non-zero (alert)
Meaning: A PCR changed, IMA count regressed/violations increased, kdump readiness was lost, or the baseline signature failed.

Fix: A changed PCR on a host that was not reinstalled is a strong boot-tamper indicator. Investigate before trusting the host.

Deployment

Record the baseline on a trusted boot (ideally right after provisioning, as root so IMA is captured), store it signed, and run attest verify --pubkey early in boot or from a timer. Route an alert to the console.

Honest limits

- This reads PCRs from sysfs; it does not perform a remote attestation quote (TPM-signed TPM2_Quote with a nonce).
- A fully compromised kernel can lie to /proc/sys; cross-view and TPM/IMA are the hedge, not a guarantee.
- Forcing a kdump capture on tamper is a privileged, destructive action left to the operator's response policy.

CAU engine commands

Every command accepts --help. Flags take one dash or two and can go before or after paths.

COMMAND	WHAT IT DOES
cau scan [flags] [path ...]	Scan files and directories with every configured detector
cau ps [flags]	Scan the live process table over /proc; --respond stops confirmed processes
cau net [flags]	Scan the network plane; block|unblock|blocks|clear for reversible drop rules
cau persist [flags]	Audit the host for persistence (read-only)
cau defend [flags]	Chain every layer: incidents, real-time file scanning, automatic response
cau installed [flags]	Audit already-installed packages' maintainer scripts
cau firmware [flags]	Parse a UEFI firmware image: inventory, FV integrity, rules
cau boot-scan [flags]	Read the boot chain, read-only: MBR/GPT, ESP, Secure Boot, firmware tables
cau hwid [flags]	Snapshot the hardware identity; --check compares against a baseline
cau peel [flags] file...	Walk obfuscation chains: every common base, rot and Caesar, in parallel
cau feed subcommand [flags]	Abuse-intel feed: list / check / update
cau intake --format fmt [flags]	Ingest STIX, MISP, MalwareBazaar, LOLDrivers or TAXII 2.1 intel
cau update (--from|--url) [flags]	Install a signed signature-DB bundle, refusing an older one
cau selftest [--record] [flags]	Verify detection content, feed, config and binary against a baseline
cau verify-self (--from|--url)	Verify this binary against the signed release manifest
cau bundle update|checksums|sbom	Produce signed release artifacts and a CycloneDX SBOM
cau quarantine subcommand	Vault flagged files: add / list / restore / verify / delete
cau canary plant|check [flags]	Plant decoy files and detect tampering
cau hash [flags] path...	Print SHA-256, imphash, TLSH and ssdeep digests
cau cluster [flags] path...	Group PE and ELF files by import-table similarity
cau rules lint|list [flags]	Lint the YARA rules, or list them with their meta
cau coverage [flags]	ATT&CK coverage matrix of the shipped content
cau honesty [flags]	What is covered, what was measured, whether the content is intact
cau version / cau help / cau completion shell	Version / help / shell completion (bash, zsh, fish)

cau scan detector flags

FLAG	DEFAULT	MEANING
--no-yara	off	Disable YARA
--no-heuristics	off	Disable string and API heuristics
--no-static	off	Disable the static structural pass
--no-fuzzy	off	Keep static but skip TLSH/ssdeep (faster on large files)
--signers file	off	Known-abused code-signing cert blocklist (lolcerts)
--no-stego	off	Disable invisible-codepoint pass
--no-encchain	off	Disable encoded-executable pass
--no-lnk	off	Disable Windows Shell Link (.lnk) pass
--no-ole	off	Disable OLE compound-file (VBA macro) pass
--no-pdf	off	Disable malicious-document pass
--no-iso	off	Disable ISO 9660 container pass
--no-pkg	off	Disable Linux software-package pass
--no-archive	off	Disable compressed-container pass (ZIP/TAR/gzip)
--no-chm	off	Disable CHM help-file pass
--no-tradecraft	off	Disable behavioural pass over scripts and payload bytes
--no-peel	off	Disable obfuscation-chain peeler
--no-fusion	off	Disable fusion lattice
--heuristic-threshold N	7	Heuristic score for SUSPICIOUS
--static-threshold N	8	Static anomaly score for SUSPICIOUS
--stego-threshold N	4	Stego score for SUSPICIOUS
--encchain-threshold N	8	Encoded-executable score for SUSPICIOUS
--fail-on level	—	CI failure threshold: none, heuristic, suspicious, malicious

Common scan flags

FLAG	MEANING
--file PATH	Single file (positional paths accepted too)
--dir PATH	Directory to scan recursively (also sets SARIF source root)
--db PATH	Signature database (default: auto-discovered)
--rules PATH	YARA rule file or directory (default: auto-discovered)
--fuzzy-db PATH	Fuzzy signature database (default: auto-discovered, optional)
--format fmt	Output format: text, json, sarif, ocsf, syslog, cef, leef, otlp, ecs, stix
--output PATH	Write report to file instead of stdout
--jobs N	Parallel workers for directory scans
--quiet	Suppress banner and progress (stderr)
--respond stop|quarantine|block-exe|netblock	Response action on confirmed findings
--report-only	Watch without changing anything

Exit codes

CODE	MEANING
0	Clean — no findings at the --fail-on level
1	Findings at the --fail-on level
2	Unscannable files
3	The run failed
4	Usage error

Detection content & output formats

Fifteen file detectors, three runtime planes, and a fusion lattice. Thirteen output formats including OCSF, SARIF, CEF, LEEF, syslog, OTLP, ECS and STIX.

File detectors (fifteen, single-pass)

- Hash signatures — exact content known-malicious
- YARA (in-process libyara, per-file timeout)
- Heuristics — suspicious capability naming
- Static structure — PE, ELF, Mach-O parsers, entropy, TLSH/ssdeep/imphash/telfhash
- Invisible codepoints (stego)
- Encoded executable (base64/hex multi-layer)
- Obfuscation-chain peeler
- Shell Link (.lnk)
- OLE macro (VBA)
- PDF document
- ISO container
- Linux software package (.deb/.rpm/Arch/Flatpak/AppImage/Docker/Snap)
- Archive (ZIP/TAR/gzip)
- CHM help file
- Tradecraft (behaviour in scripts and payload bytes)
- Fusion — cross-detector corroboration

Runtime planes

PLANE	COMMAND	QUESTION IT ANSWERS
Process	cau ps	Is anything running that a benign system would not run? (fileless, masquerade, injection, network shells)
Network	cau net	What is this host talking to, and who on this host is doing it? Every socket attributed to its process and file.
Persistence	cau persist	What will run again after a reboot or a login?

cau defend joins all three planes with the file detectors into one incident. Response is tiered: --respond stop freezes a confirmed process, --quarantine seals a file, --block-exe asks the kernel guard to refuse it, --netblock drops its peers in cau's own nftables table.

Common errors & fixes

A consolidated reference for errors you may encounter and how to resolve them.

ERROR	MEANING	FIX
package yara was not found in the pkg-config search path	pkg-config cannot see libyara	Install libyara-dev; source scripts/dev-env.sh or export PKG_CONFIG_PATH
make: No rule to make target 'build'	Not in repo root / Makefile missing	cd into cau; confirm Makefile exists
go: command not found	Go not installed or not on PATH	Install Go; verify with go version
cau: command not found	Binary not installed or not on PATH	Run make install; add $HOME/.local/bin to PATH if user-local
signature database not found (fatal)	No hashes.txt in discovery order	Pass --db explicitly or set CAU_SIGNATURES env var
operation not permitted (enforce run)	Missing CAP_BPF + CAP_MAC_ADMIN or root	Run with sudo or setcap cap_bpf,cap_mac_admin+ep
bpf not in /sys/kernel/security/lsm	Kernel lacks BPF-LSM	Use kmod backend or rebuild kernel with CONFIG_BPF_LSM
insmod: Required key not available	Secure Boot blocks unsigned module	Sign module with enrolled MOK key or disable Secure Boot
control map not found / map not pinned	enforce run not executed or guard died	Start guard with enforce run first
baseline signature does not verify	Baseline tampered or wrong pubkey	Regenerate baseline with matching sign-key/pubkey
drift exit code non-zero	Security-relevant change since baseline	Review output; contain host if unexpected
rmmod: Module is in use	Module still referenced	Set enforce to 0 first, then rmmod
--memfd deny breaks runc	runc re-executes through memfd	Keep --memfd audit unless no container runtime

Honest limits & disclaimer

What the software cannot do, and the scope of this guide.

Kernel enforcement limits

- BPF-LSM needs CONFIG_BPF_LSM and bpf in /sys/kernel/security/lsm; not all kernels have it.
- Both backends are fail-open — a crash lifts the freeze, by design.
- Network-shell guard blind spots: shells on a pty, pipe relays, and shells started first then connected by an in-process implant.
- Killguard refuses PID 1 and its own PID; it is not itself protected.
- A fully compromised kernel can lie to /proc/sys; cross-view and TPM/IMA are the hedge.

Disclaimer

For educational and research use. Only analyze files and systems you own or are authorised to analyze. Handle real malware samples in isolated environments and never commit them to this repository. Authorized testing only — written authorization is required before any assessment begins.